CS 234 Winter 2026
Assignment 1
Due: January 16th 2026 at 6:00 pm (PST)
Sunid:

Submission Instructions (Please follow these exactly)

Please typeset the written part of your homework using the template that we have provided in Latex. You will need to submit the homework in three separate parts:

- **Assignment 1 Written (PDF file):** Submit a PDF version of your LaTeX file.
- **Assignment 1 Written (Tex file):** Submit the raw `.tex` file.
- **Assignment 1 Coding (.zip file):** Submit your coding solutions here. There is a `Makefile` provided that will help you submit the assignment. Please run `make clean` followed by `make submit` in the terminal and submit the resulting zip file on Gradescope.

1 Effect of Effective Horizon [8 pts]

Consider an agent managing inventory for a store, which is represented as an MDP. The stock level s refers to the number of items currently in stock (between 0 and 10, inclusive). At any time, the agent has two actions: sell (decrease stock by one, if possible) or buy (increase stock by one, if possible).

- If $s > 0$ and the agent sells, it receives $+1$ reward for the sale and the stock level transitions to $s - 1$. If $s = 0$ nothing happens.
- If $s < 9$ and the agent buys, it receives no reward and the stock level transitions to $s + 1$.
- The owner of the store likes to see a fully stocked inventory at the end of the day, so the agent is rewarded with $+100$ if the stock level ever reaches the maximum level $s = 10$.
- $s = 10$ is also a terminal state and the problem ends if it is reached.

The reward function, denoted as $r(s, a, s')$, can be summarized concisely as follows:

- $r(s, \text{sell}, s - 1) = 1$ for $s > 0$ and $r(0, \text{sell}, 0) = 0$
- $r(s, \text{buy}, s + 1) = 0$ for $s < 9$ and $r(9, \text{buy}, 10) = 100$. The last condition indicates that transitioning from $s = 9$ to $s = 10$ (fully stocked) yields $+100$ reward.

The stock level is assumed to always start at $s = 3$ at the beginning of the day. We will consider how the agent's optimal policy changes as we adjust the finite horizon H of the problem. Recall that the horizon H refers to a limit on the number of time steps the agent can interact with the MDP before the episode terminates, regardless of whether it has reached a terminal state. We will explore properties of the optimal policy (the policy that achieves highest episode reward) as the horizon H changes.

Consider, for example, $H = 4$. The agent can sell for three steps, transitioning from $s = 3$ to $s = 2$ to $s = 1$ to $s = 0$ receiving rewards $+1$, $+1$, and $+1$ for each sell action. At the fourth step, the inventory is empty so it can sell or buy, receiving no reward regardless. Then the problem terminates since time has expired.

(a) Starting from the initial state $s = 3$, it possible to a choose a value of H that results in the optimal policy taking both buy and sell steps during its execution? Explain why or why not. [2 pts]

Yes. For example, let $H = 5$. The action sequence sell, sell, sell, buy, sell takes the inventory through $3 \to 2 \to 1 \to 0 \to 1 \to 0$ and earns a total undiscounted reward of 4. Reaching $s = 10$ requires at least 7 steps, so the 100 reward is unattainable. Furthermore, five rewarded sales in five steps are impossible with only three items initially in stock. Thus, 4 is the maximum achievable reward, and this optimal sequence includes both buy and sell actions.

For $H \geq 9$, an optimal trajectory can also combine sales with reaching full inventory. Let $k = \lfloor (H-7)/2 \rfloor \geq 1$. The agent can perform k buy–sell pairs, returning to $s = 3$ after each pair, and then buy seven times to reach $s = 10$. This earns $100 + k$ in $2k + 7 \leq H$ steps. Any trajectory reaching $s = 10$ with m sales needs $m + 7$ buys, so it can earn at most $100 + k$. A trajectory not reaching $s = 10$ can earn at most $\lfloor (H+3)/2 \rfloor = k+5$ from sales. Thus, the constructed trajectory is optimal and includes both actions, with all buy–sell pairs completed before termination.

(b) In the infinite-horizon discounted setting, is it possible to choose a fixed value of $\gamma \in [0, 1)$ such that the optimal policy starting from $s = 3$ never fully stocks the inventory? You do not need to propose a specific value, but simply explain your reasoning either way. [2 pts]

Yes. There are two cases. Starting from $s = 3$, reaching $s = 10$ requires seven consecutive buys, so the 100 reward received on the seventh action contributes $100\gamma^6$, since the first action's reward is undiscounted. Selling on the first action instead contributes 1. For sufficiently small positive γ, $100\gamma^6 < 1$, so immediate selling is preferable to buying directly to full inventory. The agent therefore sells down to $s = 0$ and then repeatedly buys and sells between $s = 0$ and $s = 1$, never reaching full inventory.

When γ is sufficiently close to 1, avoiding termination is also optimal. At $s = 9$, buying immediately yields 100, while selling, buying back to $s = 9$, and then buying to $s = 10$ yields $1 + 100\gamma^2$: the sale reward is received on the first action and the terminal reward on the third. For γ close enough to 1, $1 + 100\gamma^2 > 100$. Under this condition, replacing the final buy from $s = 9$ with sell, buy, buy strictly increases the total discounted return. Thus, no trajectory reaching $s = 10$ can be optimal, and an optimal policy never fully stocks the inventory.

(c) Consider two versions of this inventory MDP. In the first version, the MDP is an *infinite-horizon* MDP with discount factor γ. In the second version, the MDP is a *finite-horizon* MDP with horizon H, no discount factor, and episodes that terminate after exactly H time steps even if a terminal state has not been reached.

Does there **ever** exist a choice of γ such that the optimal policy for the infinite-horizon MDP is the same as the optimal policy for the finite-horizon MDP with horizon H? If so, give a concrete example of values of γ and H for which this holds. [2 pts]

Yes. Choose $\gamma = 0.9$ and $H = 7$. In the finite-horizon case, seven consecutive buys reach $s = 10$ and earn 100. Any other action sequence cannot reach $s = 10$ within seven steps and earns at most 7 from sales, so buying at every step is optimal.

In the infinite-horizon case, seven consecutive buys give a discounted return of $100(0.9)^6 = 53.1441$. Any trajectory that includes at least one sale and eventually reaches $s = 10$ requires at least eight buys and one sale, so its terminal reward is received no earlier than the ninth action. Its discounted terminal reward is therefore at most $100(0.9)^8 = 43.046721$. The total discounted reward from sales is at most $1 + 0.9 + 0.9^2 + \cdots = 1/(1 - 0.9) = 10$, even if a sale reward were received at every step. Thus, any such trajectory has return at most

$$10 + 43.046721 = 53.046721 < 53.1441.$$

A trajectory that never reaches full inventory earns at most 10. Hence, starting from $s = 3$, buying seven times until termination is optimal in both settings.

(d) Using the same setup as in part (c), does there **always** exist a discount factor γ such that the optimal policy for the infinite-horizon MDP matches the optimal policy for the finite-horizon MDP for any H? Briefly justify your answer in 1–2 sentences.[2 pts]

No: although finite-horizon optimal trajectories need not be unique, for $H = 9$ every optimal trajectory uses one sale and eight buys. For every $0 < \gamma < 1$, even the best ordering (sell once, then buy eight times) is strictly worse than either buying seven times directly or selling twice and then buying nine times, since

$$\max\left\{100\gamma^6,\ 1 + \gamma + 100\gamma^{10}\right\} > 1 + 100\gamma^8;$$

at $\gamma = 0$, it also fails to be optimal at every visited state, since buying at $s = 2$ yields 0 while selling yields 1.

2 Reward Hacking [5 pts]

Q1 illustrates how the particular horizon and discount factor may lead to to very different policies, even with the same reward and dynamics model. This may lead to unintentional reward hacking, where the resulting policy does not match a human stakeholder's intended outcome. This problem asks you to think about an example where reward hacking may occur, introduced by Pan, Bhatia and Steinhardt[1]. Consider designing RL for autonomous cars where the goal is to have decision policies that minimize the mean commute for all drivers (those driven by humans and those driven by AI). This reward might be tricky to specify (it depends on the destination of each car, etc) but a simpler reward (called the reward "proxy") is to maximize the mean velocity of all cars. Now consider a scenario where there is a single AI car (the red car in the figure) and many cars driven by humans (the grey car).

In this setting, under this simpler "proxy" reward, the optimal policy for the red (AI) car is to park and not merge onto the highway.[2]

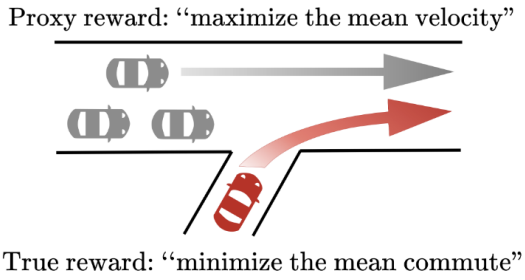


Figure 1: Pan, Bhatia, Steinhardt ICLR 2022; https://openreview.net/pdf?id=JYtwGwIL7ye

(a) Explain why the optimal policy for the AI car is not to merge onto the highway. [2 pts]

> Because there are many human-driven cars, the red car's merging would cause many of them to slow down. The increase in the red car's speed would not compensate for the combined decrease in their speeds, so remaining parked yields a higher mean velocity and hence a higher proxy reward.

[1] ICLR 2022 https://openreview.net/pdf?id=JYtwGwIL7ye

[2] Interestingly, it turns out that systems that use simpler function representations may reward hack less in this example than more complex representations. See Pan, Bhatia and Steinhardt's paper "The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models" for details.

(b) Note this behavior is not aligned with the true reward function. Share some ideas about alternate reward functions (that are not minimizing commute) that might still be easier to optimize, but would not result in the AI car never merging. Your answer should be 2-5 sentences and can include equations: there is not a single answer and reasonable solutions will be given full credit. [3 pts]

Keep the mean-velocity reward and apply the following penalties at every step:

$$\begin{aligned} r_t = \bar{v}_t &- \lambda_1 \mathbf{1}\{\text{AI on ramp},\ v_t^{\text{AI}} < v_{\min}\} \\ &- \lambda_2 \sum_{i \in \mathcal{M}_t} \mathbf{1}\{d_i < d_{\text{safe}}\} \frac{\max(v_i^{\text{rear}} - v_i^{\text{front}}, 0)}{\max(d_i, \varepsilon)} \\ &- \lambda_3 \sum_{i \in \mathcal{M}_t} \mathbf{1}\{d_i < d_{\text{critical}}\}. \end{aligned}$$

Here, $\bar{v}_t$ is the mean velocity, $\mathcal{M}_t$ contains adjacent rear–front vehicle pairs on the main road, d_i is their gap, $0 < d_{\text{critical}} < d_{\text{safe}}$, $\varepsilon > 0$ prevents division by zero, $\mathbf{1}$ is an indicator, and the $\lambda_j > 0$ control the penalty strengths.

1. The λ_1 term penalizes low speed on the ramp at every step, discouraging the AI car from remaining parked instead of merging.
2. The λ_2 term penalizes a rear vehicle approaching its front vehicle when their gap is below d_{safe}, with a larger penalty for faster closing or a smaller gap.
3. The λ_3 term applies a strong penalty below d_{critical} even at equal speeds, discouraging dangerously close following that the relative-speed term would miss.

3 Bellman Residuals and performance bounds [30 pts]

In this problem, we will study value functions and properties of the Bellman backup operator.

Definitions: Recall that a value function is a $|S|$-dimensional vector where $|S|$ is the number of states of the MDP. When we use the term V in these expressions as an "arbitrary value function", we mean that V is an arbitrary $|S|$-dimensional vector which need not be aligned with the definition of the MDP at all. On the other hand, V^π is a value function that is achieved by some policy π in the MDP. For example, say the MDP has 2 states and only negative immediate rewards. $V = [1, 1]$ would be a valid choice for V even though this value function can never be achieved by any policy π, but we can never have a $V^\pi = [1, 1]$. This distinction between V and V^π is important for this question and more broadly in reinforcement learning.

Properties of Bellman Operators: In the first part of this problem, we will explore some general and useful properties of the Bellman backup operator, which was introduced during lecture. We know that the Bellman backup operator B, defined below is a contraction with the fixed point as V^*, the optimal value function of the MDP. The symbols have their usual meanings. γ is the discount factor and $0 \leq \gamma < 1$. In all parts, $\|v\| = \max_s |v(s)|$ is the infinity norm of the vector.

$$(BV)(s) = \max_a \left(r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a)V(s') \right)$$

We also saw the contraction operator B^π with the fixed point V^π, which is the Bellman backup operator for a particular policy given below:

$$(B^\pi V)(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s))V(s')$$

In this case, we'll assume π is deterministic, but it doesn't have to be in general. In class, we showed that $||BV - BV'|| \leq \gamma||V - V'||$ for two arbitrary value functions V and V'.

(a) Show that the analogous inequality, $||B^\pi V - B^\pi V'|| \leq \gamma||V - V'||$, also holds. [3 pts].

For any $s \in S$, the immediate reward terms cancel, so

$$\begin{aligned}
\left|(B^\pi V - B^\pi V')(s)\right| &= \gamma \left| \sum_{s' \in S} p(s' \mid s, \pi(s)) \big(V(s') - V'(s')\big) \right| \\
&\leq \gamma \sum_{s' \in S} p(s' \mid s, \pi(s)) \left|V(s') - V'(s')\right| \\
&\leq \gamma \|V - V'\|_\infty \sum_{s' \in S} p(s' \mid s, \pi(s)) = \gamma \|V - V'\|_\infty,
\end{aligned}$$

where we used nonnegativity of the transition probabilities and $\sum_{s' \in S} p(s' \mid s, \pi(s)) = 1$. Taking the maximum over $s \in S$ gives

$$\|B^\pi V - B^\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty.$$

(b) Prove that the fixed point for B^π is unique. Recall that the fixed point is defined as V satisfying $V = B^\pi V$. You may assume that a fixed point exists. *Hint:* Consider proof by contradiction. [3 pts].

Suppose, for contradiction, that B^π has two distinct fixed points $V \neq V'$, so $V = B^\pi V$ and $V' = B^\pi V'$. By part (a),

$$\|V - V'\|_\infty = \|B^\pi V - B^\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty.$$

Since $V \neq V'$, we have $\|V - V'\|_\infty > 0$. Dividing by this norm gives $1 \leq \gamma$, contradicting $\gamma < 1$. Therefore, the fixed point of B^π is unique.

(c) Suppose that V and V' are vectors satisfying $V(s) \leq V'(s)$ for all s. Show that $B^\pi V(s) \leq B^\pi V'(s)$ for all s. Note that all of these inequalities are element-wise. [3 pts].

For any $s \in S$,

$$\begin{aligned}(B^\pi V')(s) &= r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s' \mid s, \pi(s)) V'(s') \\ &\geq r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s' \mid s, \pi(s)) V(s') = (B^\pi V)(s).\end{aligned}$$

The inequality holds because $V'(s') \geq V(s')$ for every $s' \in S$, and both γ and the transition probabilities are nonnegative. Thus, $(B^\pi V)(s) \leq (B^\pi V')(s)$ for all $s \in S$.

Bellman Residuals: Having gained some intuition for value functions and the Bellman operators, we now turn to understanding how policies can be extracted and what their performance might look like. We can extract a greedy policy π from an arbitrary value function V using the equation below.

$$\pi(s) = \arg\max_a [r(s,a) + \gamma \sum_{s' \in S} p(s'|s,a)V(s')]$$

It is often helpful to know what the performance will be if we extract a greedy policy from an arbitrary value function. To see this, we introduce the notion of a Bellman residual.

Define the Bellman residual to be $(BV - V)$ and the Bellman error magnitude to be $||BV - V||$.

(d) For what value function V does the Bellman error magnitude $\|BV - V\|$ equal 0? Why? [2 pts]

The Bellman error magnitude is zero if and only if $V = V^*$, because $\|BV - V\|_\infty = 0$ is equivalent to $BV = V$, and V^* is the unique fixed point of the contraction operator B.

Lemma. B is a contraction in the infinity norm: $\|BV - BV'\|_\infty \le \gamma \|V - V'\|_\infty$.
Proof. For any $s \in S$, choose

$$a^* \in \arg\max_a \left[r(s,a) + \gamma \sum_{s' \in S} p(s' \mid s,a) V(s') \right].$$

Since $(BV')(s)$ is at least the backup value for action a^*,

$$\begin{aligned}
(BV)(s) - (BV')(s) &\le \gamma \sum_{s' \in S} p(s' \mid s, a^*) \big(V(s') - V'(s') \big) \\
&\le \gamma \sum_{s' \in S} p(s' \mid s, a^*) \left| V(s') - V'(s') \right| \\
&\le \gamma \|V - V'\|_\infty,
\end{aligned}$$

using nonnegative transition probabilities that sum to one. Interchanging V and V' gives $(BV')(s) - (BV)(s) \le \gamma \|V - V'\|_\infty$. Thus $|(BV)(s) - (BV')(s)| \le \gamma \|V - V'\|_\infty$; taking the maximum over s proves the lemma.

To see uniqueness, suppose V is another fixed point. Since $BV = V$ and $BV^* = V^*$, the lemma gives

$$\|V - V^*\|_\infty = \|BV - BV^*\|_\infty \le \gamma \|V - V^*\|_\infty.$$

As $\gamma < 1$, this forces $\|V - V^*\|_\infty = 0$, so $V = V^*$.

(e) Prove the following statements for an arbitrary value function V and any policy π. [5 pts]
Hint: Try leveraging the triangle inequality by inserting a zero term.

$$||V - V^\pi|| \leq \frac{||V - B^\pi V||}{1-\gamma}$$

$$||V - V^*|| \leq \frac{||V - BV||}{1-\gamma}$$

Method 1: Induced matrix norm. Write $B^\pi V = r_\pi + \gamma P_\pi V$ and $V^\pi = r_\pi + \gamma P_\pi V^\pi$.

$$\begin{aligned}\|V - V^\pi\|_\infty &\leq \|V - B^\pi V\|_\infty + \|B^\pi V - V^\pi\|_\infty \\ &= \|V - B^\pi V\|_\infty + \gamma \|P_\pi (V - V^\pi)\|_\infty \\ &\leq \|V - B^\pi V\|_\infty + \gamma \|V - V^\pi\|_\infty.\end{aligned}$$

where we used $\|Ax\|_\infty \leq \|A\|_\infty \|x\|_\infty$, with $\|A\|_\infty$ denoting the matrix norm induced by the vector infinity norm, defined by

$$\|A\|_\infty := \sup_{x \neq 0} \frac{\|Ax\|_\infty}{\|x\|_\infty}.$$

In particular,

$$\|P_\pi\|_\infty = \max_s \sum_{s'} |P_\pi(s, s')| = 1,$$

since P_π has nonnegative entries and $P_\pi \mathbf{1} = \mathbf{1}$. Rearranging and dividing by $1 - \gamma > 0$ gives

$$\|V - V^\pi\|_\infty \leq \frac{\|V - B^\pi V\|_\infty}{1-\gamma}.$$

Method 2: Fixed point and contraction. Using $V^\pi = B^\pi V^\pi$ and part (a),

$$\begin{aligned}\|V - V^\pi\|_\infty &\leq \|V - B^\pi V\|_\infty + \|B^\pi V - V^\pi\|_\infty \\ &= \|V - B^\pi V\|_\infty + \|B^\pi V - B^\pi V^\pi\|_\infty \\ &\leq \|V - B^\pi V\|_\infty + \gamma \|V - V^\pi\|_\infty.\end{aligned}$$

The same rearrangement proves the first bound.

For the optimal-value bound, use $V^* = BV^*$ and the contraction property of B:

$$\begin{aligned}\|V - V^*\|_\infty &\leq \|V - BV\|_\infty + \|BV - V^*\|_\infty \\ &= \|V - BV\|_\infty + \|BV - BV^*\|_\infty \\ &\leq \|V - BV\|_\infty + \gamma \|V - V^*\|_\infty.\end{aligned}$$

Thus $(1-\gamma)\|V - V^*\|_\infty \leq \|V - BV\|_\infty$, yielding

$$\|V - V^*\|_\infty \leq \frac{\|V - BV\|_\infty}{1-\gamma}.$$

The result you proved in part (e) will be useful in proving a bound on the policy performance in the next few parts. Given the Bellman residual, we will now try to derive a bound on the policy performance, V^π.

(f) Let V be an arbitrary value function and π be the greedy policy extracted from V. Let $\varepsilon = ||BV - V||$ be the Bellman error magnitude for V. Prove the following for any state s. [5 pts]
Hint: Try to use the results from part (e).

$$V^\pi(s) \geq V^*(s) - \frac{2\varepsilon}{1-\gamma}$$

Since π is greedy with respect to V, we have $B^\pi V = BV$. By part (e),

$$\|V - V^\pi\|_\infty \leq \frac{\|V - B^\pi V\|_\infty}{1-\gamma} = \frac{\varepsilon}{1-\gamma}, \qquad \|V - V^*\|_\infty \leq \frac{\|V - BV\|_\infty}{1-\gamma} = \frac{\varepsilon}{1-\gamma}.$$

Hence, by the triangle inequality,

$$\begin{aligned}\|V^* - V^\pi\|_\infty &\leq \|V^* - V\|_\infty + \|V - V^\pi\|_\infty \\ &\leq \frac{2\varepsilon}{1-\gamma}.\end{aligned}$$

Therefore, for every state s,

$$V^*(s) - V^\pi(s) \leq |V^*(s) - V^\pi(s)| \leq \frac{2\varepsilon}{1-\gamma},$$

which gives $V^\pi(s) \geq V^*(s) - 2\varepsilon/(1-\gamma)$.

(g) Give an example real-world application or domain where having a lower bound on $V^{\pi}(s)$ would be useful. [2 pt]

In warehouse robot navigation, rewards can represent the time saved when transporting goods. A lower bound on $V^{\pi}(s)$ helps determine whether the robot's routing policy is close enough to optimal, allowing us to stop training and save computational cost.

(h) Suppose we have another value function V' and extract its greedy policy π'. $\|BV' - V'\| = \varepsilon = \|BV - V\|$. Does the above lower bound imply that $V^{\pi}(s) = V^{\pi'}(s)$ at any s? [2 pts]

No. The same Bellman error gives the same performance lower bound, but does not imply that the policy values are equal. By the triangle inequality and part (f),

$$\|V^{\pi} - V^{\pi'}\|_{\infty} \leq \|V^{\pi} - V^{*}\|_{\infty} + \|V^{\pi'} - V^{*}\|_{\infty} \leq \frac{4\varepsilon}{1-\gamma}.$$

This bounds their difference, but does not force it to be zero.

A little bit more notation: define $V \leq V'$ if $\forall s$, $V(s) \leq V'(s)$.

What if our algorithm returns a V that satisfies $V^* \leq V$? I.e., it returns a value function that is better than the optimal value function of the MDP. Once again, remember that V can be any vector, not necessarily achievable in the MDP but we would still like to bound the performance of V^π where π is extracted from said V. We will show that if this condition is met, then we can achieve an even tighter bound on policy performance.

(i) Using the same notation and setup as part (e), if $V^* \leq V$, show the following holds for any state s. [5 pts]
Hint: Recall that $\forall \pi$, $V^\pi \leq V^*$. (why?)

$$V^\pi(s) \geq V^*(s) - \frac{\varepsilon}{1-\gamma}$$

where $\varepsilon = \|BV - V\|$ (as above) and the policy π is the greedy policy induced by V.

First, for any policy π, let $d = V^* - V^\pi$. Since $V^* = BV^* \geq B^\pi V^*$ and $V^\pi = B^\pi V^\pi$,

$$d \geq B^\pi V^* - B^\pi V^\pi = \gamma P_\pi d.$$

Thus $(I - \gamma P_\pi)d \geq 0$. The inverse $(I - \gamma P_\pi)^{-1} = \sum_{k=0}^{\infty} \gamma^k P_\pi^k$ has nonnegative entries, so $d \geq 0$, proving $V^\pi \leq V^*$.
Now, since π is greedy with respect to V, $BV = B^\pi V$, and hence

$$BV - V^\pi = \gamma P_\pi (V - V^\pi).$$

Using the triangle inequality and $\|P_\pi\|_\infty = 1$,

$$\begin{aligned} \|V - V^\pi\|_\infty &\leq \|V - BV\|_\infty + \|BV - V^\pi\|_\infty \\ &\leq \varepsilon + \gamma \|V - V^\pi\|_\infty. \end{aligned}$$

Therefore $\|V - V^\pi\|_\infty \leq \varepsilon/(1-\gamma)$. Since $V^\pi \leq V^* \leq V$, for every state s,

$$0 \leq V^*(s) - V^\pi(s) \leq V(s) - V^\pi(s) \leq \frac{\varepsilon}{1-\gamma}.$$

Thus $V^\pi(s) \geq V^*(s) - \varepsilon/(1-\gamma)$, as required.

Intuition: A useful way to interpret the results from parts (h) (and (i)) is based on the observation that a constant immediate reward of r at every time-step leads to an overall discounted reward of $r + \gamma r + \gamma^2 r + \ldots = \frac{r}{1-\gamma}$. Thus, the above results say that a state value function V with Bellman error magnitude ε yields a greedy policy whose reward per step (on average), differs from optimal by at most 2ε. So, if we develop an algorithm that reduces the Bellman residual, we're also able to bound the performance of the policy extracted from the value function outputted by that algorithm, which is very useful!

Challenges: Try to prove the following if you're interested. **These parts will not be graded.**

(j) It's not easy to show that the condition $V^* \leq V$ holds because we often don't know V^* of the MDP. Show that if $BV \leq V$ then $V^* \leq V$. Note that this sufficient condition is much easier to check and does not require knowledge of V^*.
Hint: Try to apply induction. What is $\lim_{n\to\infty} B^n V$?

First, B is monotone. If $V_1 \leq V_2$, choose a policy π that is greedy with respect to V_1. Since $\gamma \geq 0$ and P_π has nonnegative entries,

$$BV_1 = B^\pi V_1 = r_\pi + \gamma P_\pi V_1 \leq r_\pi + \gamma P_\pi V_2 = B^\pi V_2 \leq BV_2.$$

Thus $V_1 \leq V_2$ implies $BV_1 \leq BV_2$.
Now the assumption gives $BV \leq V$. If $B^{n+1}V \leq B^n V$, applying monotonicity yields $B^{n+2}V \leq B^{n+1}V$. Hence, by induction,

$$V \geq BV \geq B^2 V \geq \cdots \geq B^n V \qquad \text{for every } n \geq 0.$$

Since $BV^* = V^*$ and B is a γ-contraction,

$$\|B^n V - V^*\|_\infty = \|B^n V - B^n V^*\|_\infty \leq \gamma^n \|V - V^*\|_\infty \longrightarrow 0.$$

Therefore $B^n V \to V^*$. Taking the limit in $B^n V(s) \leq V(s)$ for each state s gives $V^*(s) \leq V(s)$, so $V^* \leq V$.

(k) It is possible to make the bounds from parts (f) and (i) tighter. Let V be an arbitrary value function and π be the greedy policy extracted from V. Let $\varepsilon = ||BV - V||$ be the Bellman error magnitude for V. Prove the following for any state s:

$$V^{\pi}(s) \geq V^{*}(s) - \frac{2\gamma\varepsilon}{1-\gamma}$$

Further, if $V^{*} \leq V$, prove for any state s

$$V^{\pi}(s) \geq V^{*}(s) - \frac{\gamma\varepsilon}{1-\gamma}$$

Since π is greedy with respect to V, $BV = B^{\pi}V$. Using $V^{*} = BV^{*}$ and $V^{\pi} = B^{\pi}V^{\pi}$,

$$\begin{aligned}\|V^{*} - V^{\pi}\|_{\infty} &\leq \|V^{\pi} - B^{\pi}V\|_{\infty} + \|BV - BV^{*}\|_{\infty} \\ &\leq \gamma\|V^{\pi} - V\|_{\infty} + \gamma\|V - V^{*}\|_{\infty}.\end{aligned}$$

The first inequality is the triangle inequality; the second uses $\|V^{\pi} - B^{\pi}V\|_{\infty} = \|B^{\pi}V^{\pi} - B^{\pi}V\|_{\infty}$ and the contraction properties of B^{π} and B. By part (e), both $\|V^{\pi} - V\|_{\infty}$ and $\|V - V^{*}\|_{\infty}$ are at most $\varepsilon/(1-\gamma)$. This proves $V^{\pi}(s) \geq V^{*}(s) - 2\gamma\varepsilon/(1-\gamma)$ for every state s.
If additionally $V^{*} \leq V$, monotonicity gives $V^{*} = BV^{*} \leq BV$. Since $V^{\pi} \leq V^{*}$, for every state s,

$$\begin{aligned}0 \leq V^{*}(s) - V^{\pi}(s) &\leq (BV)(s) - V^{\pi}(s) \\ &= \gamma\big(P_{\pi}(V - V^{\pi})\big)(s) \\ &\leq \gamma\|V - V^{\pi}\|_{\infty} \\ &\leq \frac{\gamma\varepsilon}{1-\gamma},\end{aligned}$$

where the second inequality uses the nonnegative entries and unit row sums of P_{π}. Hence $V^{\pi}(s) \geq V^{*}(s) - \gamma\varepsilon/(1-\gamma)$, as required.

4 RiverSwim MDP [25 pts]

Now you will implement value iteration and policy iteration for the RiverSwim environment (see picture below[3]) of (Strehl & Littman, 2008).

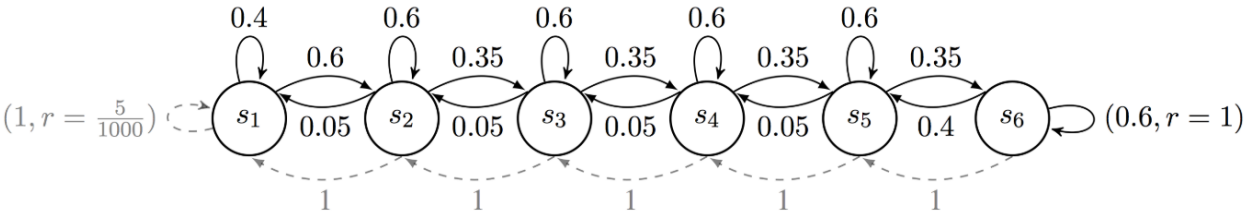


Figure 2: The RiverSwim MDP where dashed and solid arrows represent transitions for the LEFT and RIGHT actions, respectively. The assignment uses a modified, customizable version of what is shown above where there are three different strengths (WEAK, MEDIUM, or STRONG) of the current (transition probabilities for being pushed back or successfully swimming RIGHT).

Setup: This assignment needs to be completed with Python3 and `numpy`.

Submission: There is a `Makefile` provided that will help you submit the assignment. Please run `make clean` followed by `make submit` in the terminal and submit the resulting zip file on Gradescope.

(a) **(coding)** Read through `vi_and_pi.py` and implement `bellman_backup`. Return the value associated with a single Bellman backup performed for an input state-action pair. [4 pts]

(b) **(coding)** Implement `policy_evaluation`, `policy_improvement` and `policy_iteration` in `vi_and_pi.py`. Return the optimal value function and the optimal policy. [8pts]

(c) **(coding)** Implement `value_iteration` in `vi_and_pi.py`. Return the optimal value function and the optimal policy. [8 pts]

(d) **(written)** Run both methods on RiverSwim with a WEAK current strength and find the largest discount factor (**only** up to two decimal places) such that an optimal agent starting in the initial far-left state (state s_1 in Figure 2) **does not** swim up the river (that is, does not go RIGHT). Using the value you find, interpret why this behavior makes sense. Now repeat this for RiverSwim with MEDIUM and STRONG currents, respectively. Describe and explain the changes in optimal values and discount factors you obtain both quantitatively and qualitatively. [5 pts]

Sanity Check: For RiverSwim with a discount factor $\gamma = 0.99$ and a WEAK current, the values for the left-most and right-most states (s_1 and s_6 in Figure 2 above) are `30.328` and `36.859` when computed with a tolerance of 0.001. The value functions from VI and PI should be within error tolerance 0.001 of these values. You can use this to verify your implementation.

[3] Figure copied from (Osband & Van Roy, 2013).

For grading purposes, we shall test your implementation against other hidden test cases as well.

Scanning $\gamma = 0.00, 0.01, \ldots, 0.99$ with both policy iteration and value iteration (tol $= 10^{-10}$) gives the same results:

Current	Largest γ with $\pi^*(s_1) = L$	Next γ with $\pi^*(s_1) = R$	$V^*(s_1)$ at $\gamma = 0.99$
Weak	0.66	0.67	30.426616
Medium	0.77	0.78	11.714942
Strong	0.93	0.94	1.194330

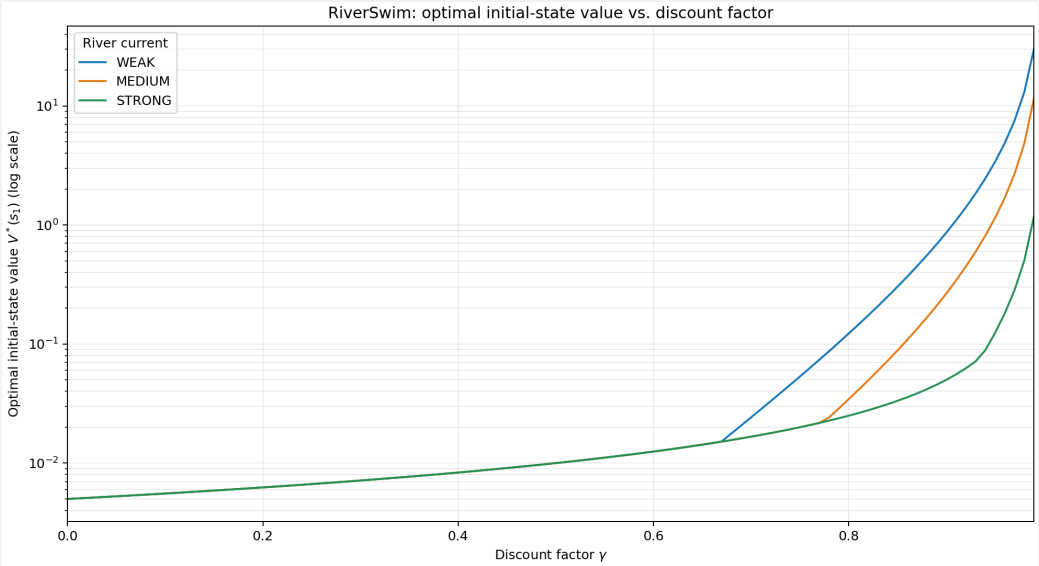


Figure 3: Optimal initial-state value $V^*(s_1)$ versus discount factor γ for three current strengths (logarithmic vertical axis). Policy iteration and value iteration agree numerically, so each current is plotted only once.

At a fixed γ, stronger currents increase the expected number of steps needed to reach the right end, reducing the expected discounted return from swimming right and making the agent more inclined to stay at the left end;
Increasing γ substantially reduces the discounting of delayed rewards, making swimming right more favorable relative to staying at the left end and thus making the agent more inclined to swim right;
Therefore, at the decision threshold, a larger γ is needed to offset the effect of a stronger current, so stronger currents correspond to higher threshold discount factors.